

Lab: Filesystem Access

Resource vs Tool for Files

This is a real design decision you will face when building MCP servers.

Resources

- Resources are passive, read-only data sources for context.
- Each has a URI and MIME type.
- Discovery uses resources/list (direct) or resources/templates/list (parameterized)
- Retrieval uses resources/read.

Tools

- Tools are model-controlled actions.
- The official Anthropic filesystem server exposes reading as tools (read_text_file, etc., all marked readOnlyHint: true), not resources, **restricted via allowed directories**.

Why Tools Instead of Pure Resources for Files?

- A resource template like **docs://{name}** matches a single URI segment — it cannot cleanly address an arbitrary nested file tree.
- The model often needs to actively decide which file to read mid-task — that is tool (model-controlled) behaviour, not passive context.

*This lab uses both deliberately: a **docs://{name}** resource template for top-level documents, plus **read_file / list_documents** tools that handle nested paths the way real filesystem servers do.*

Important: Roots are coordination, not enforcement. This server does not trust a client-provided root for security — it enforces its own ROOT containment at the OS path level.

Build a Sandboxed Filesystem Server

Step 1 — Set Up the Project and Test Directories and Files

Create the project and sample documents:

```
mkdir mcp-fs-resource
cd mcp-fs-resource
pip install mcp["cli"]
```

Create Directories and files required for testing

```
mkdir documents
mkdir documents\notes
mkdir not-in-documents

echo 'This is content of ReadMe.txt' > documents\readme.txt
echo 'This is content of TODO' > not-in-documents\todo.txt
echo 'This is content of TODO' > documents\notes\todo.txt
```

Step 2 — Create fs_server.py

```
"""Filesystem Resource Server — read-only access to documents under a sandboxed root.
```

```
Security model:
```

- * Sandbox: only files under ROOT are ever exposed.
- * Path containment: every request is resolved and checked to stay inside ROOT.
- * Read-only: no write/delete/move operations are exposed.

```
"""
```

```
import json
```

```
import os
```

```
import sys
```

```
from pathlib import Path
```

```
from mcp.server.mcpserver import MCPServer
```

```
# Sandbox root: only files under here are exposed. Override with DOCS_ROOT.
```

```
ROOT = Path(os.environ.get("DOCS_ROOT", "./documents")).resolve()
```

```
mcp = MCPServer("Filesystem Resource Server")
```

```
def _safe_path(relative: str) -> Path:
```

```
    """It returns a Path object — absolute path to the requested file or directory,
    but only if that path is safely inside ROOT."""
```

```
    absolutePath = (ROOT / relative).resolve()
```

```
    if not absolutePath.is_relative_to(ROOT):
```

```
        raise ValueError(f"Access denied: '{relative}' is outside the sandbox")
```

```
    return absolutePath
```

```

@mcp.tool()
def list_documents(subdir: str = "") -> str:
    """List readable documents under the sandbox root, recursively and paginated."""
    base = _safe_path(subdir)
    if not base.is_dir():
        raise ValueError(f"Not a directory: '{subdir}'")
    files = sorted(p for p in base.rglob("*") if p.is_file())
    total = len(files)
    items = [str(p.relative_to(ROOT)) for p in files]
    return json.dumps({"root": str(ROOT), "total": total, "documents": items})

@mcp.tool()
def read_file(filepath: str) -> str:
    """Read any document (incl. nested) by its path relative to the sandbox root."""
    target = _safe_path(filepath)
    if not target.is_file():
        raise ValueError(f"Not a file: '{filepath}'")
    return target.read_text(encoding="utf-8", errors="replace")

@mcp.resource("docs://{filename}")
def read_document(filename: str) -> str:
    """Read a top-level document as an addressable resource (read-only context)."""
    target = _safe_path(filename)
    if not target.is_file():
        raise ValueError(f"Not a file: '{filename}'")
    return target.read_text(encoding="utf-8", errors="replace")

if __name__ == "__main__":
    print(f"Serving read-only documents from {ROOT}", file=sys.stderr)
    mcp.run()

```

The server exposes two tools and one resource template, with all reads funnelled through a single security chokepoint (`_safe_path`):

- `list_documents` — lists all readable files under the sandbox root (recursive, paginated)

- `read_file` — reads any file by path relative to the sandbox root
- `read_document (resource template docs://{name})` — reads a top-level file as an addressable resource
- `_safe_path` — the path-containment guard: `resolve()` collapses `..`, `is_relative_to(ROOT)` rejects anything outside

Step 3 — Test with MCP Client

`fs_client.py`

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server = StdioServerParameters(command="python", args=["fs_server.py"])

async def main():
    async with stdio_client(server) as (r, w), ClientSession(r, w) as s:
        await s.initialize()

        # Test the resource access - All within the sandboxed root, should succeed
        readMeContent = await s.read_resource("docs://readme.txt")
        print("Resource:", readMeContent.contents[0].text)

        # Test the tools access - All within the sandboxed root, should succeed
        dirList = await s.call_tool("list_documents", arguments={"subdir": ""})
        print("List:", dirList.content[0].text)
        dirList = await s.call_tool("list_documents", arguments={"subdir": "notes"})
        print("List:", dirList.content[0].text)
        fileContent = await s.call_tool("read_file", arguments={"filepath": "readme.txt"})
        print("Read:", fileContent.content[0].text)
        fileContent = await s.call_tool("read_file", arguments={"filepath": "notes\\todo.txt"})
        print("Read:", fileContent.content[0].text)

        # Test path containment - Attempt to escape the sandbox, should fail
        dirList = await s.call_tool("list_documents", arguments={"subdir": "not-in-documents"})
        print("List:", dirList.content[0].text)
        notesContent = await s.call_tool("read_file", arguments={"filepath": "..\\not-in-documents\\todo.txt"})
```

```
print("Error:", notesContent.is_error, notesContent.content[0].text)

asyncio.run(main())
```